

Introduction to Web Service Technologies & Graphics Architectures for Non-Desktop Devices

Pablo Cesar

pcesar@tml.hut.fi

<http://www.tml.hut.fi/~pcesar>

Outline

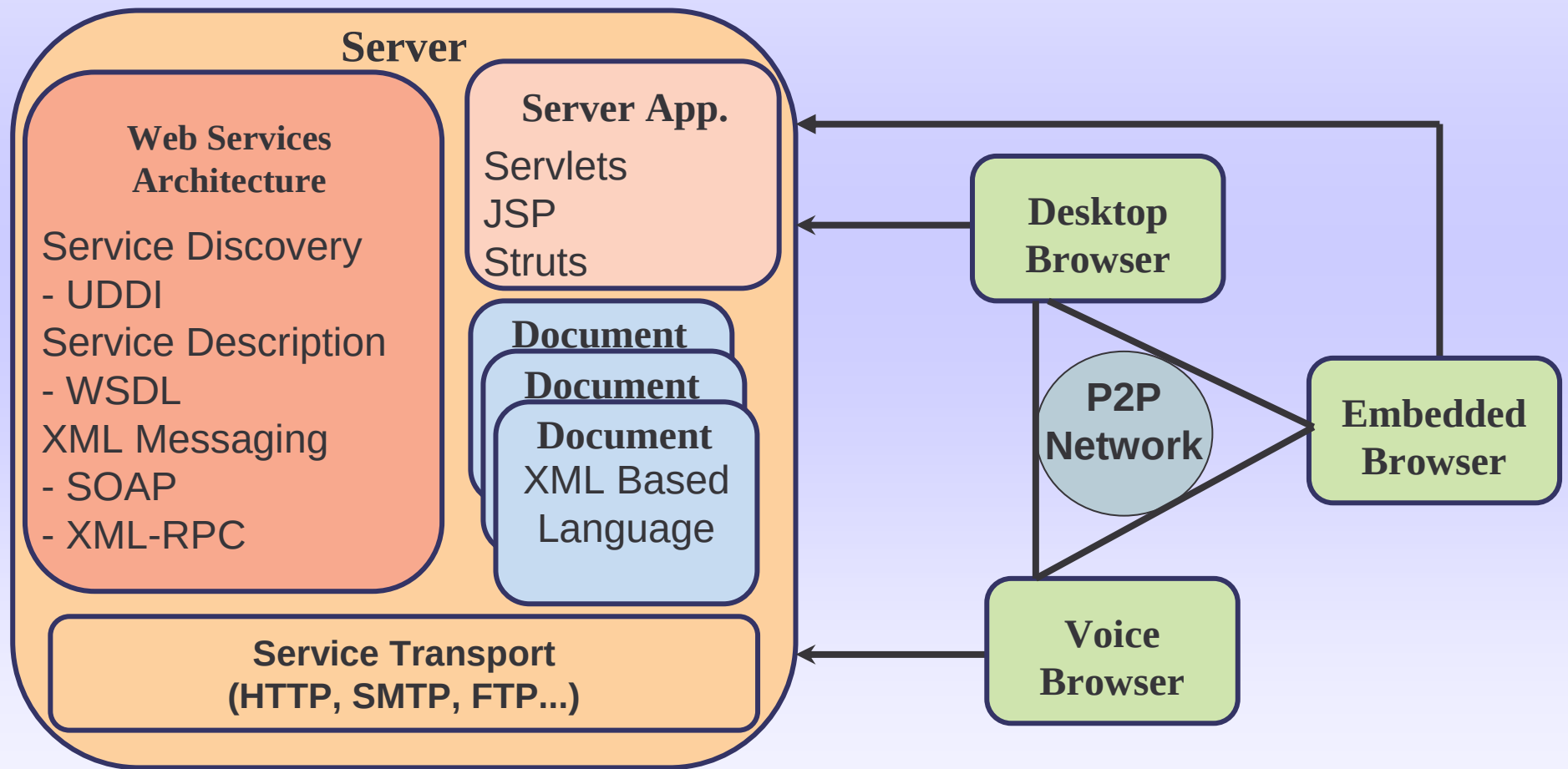
- Part I
 - Overview of the course topics
- Part II
 - Graphics Systems in Embedded Devices
- Part III
 - Experiences from a PhD Student (myself)

Outline - Part I

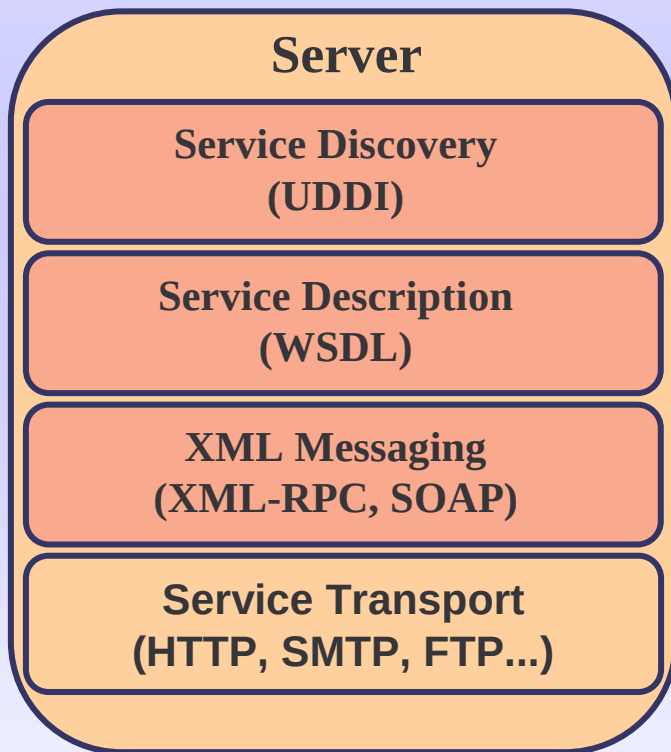
Web Service Technologies Overview

- Client/Server Architecture
 - Server Side
 - Web Services (application to application)
 - Server Applications (client human being using a browser)
 - Client Side
 - XML Based Documents
 - Terminals and Browsers
- P2P Architecture

Web Service Technologies Overview

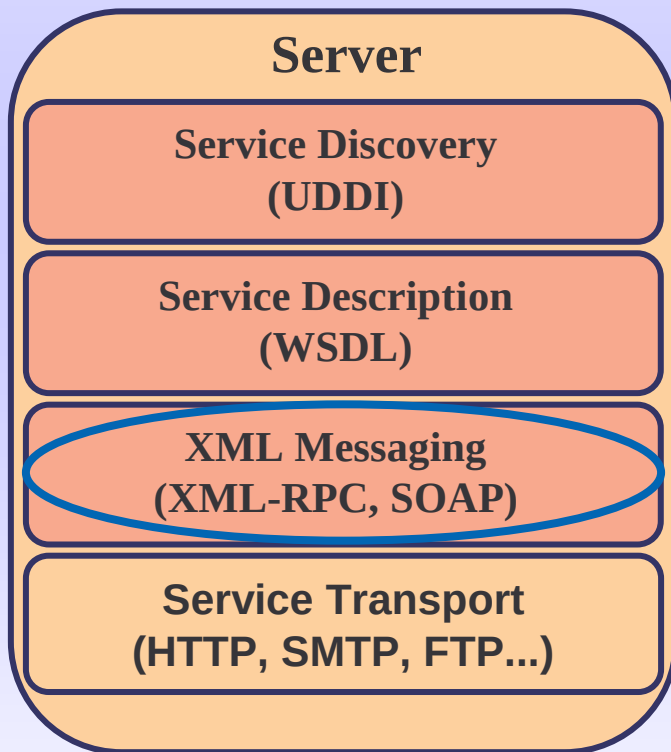


Web Services Architecture



- XML Web services:
 - Piece of Software that makes itself available over the Internet
 - Platform and Language independent (e.g., Java can talk to Perl)
- Messaging
 - Common XML format, so messages delivered among parties involved can understand each other (SOAP, XML-RC)
- Service Description
 - Describes the public interface to a Web Service (WSDL)
- Discovery
 - Centralises services in a registry. Easy publish and find (UDDI)

Web Services Architecture XML-RPC (Remote Procedure Call)



- XML-RPC = HTTP + XML + RPC
- Intended for simple applications, very easy to use

Query:

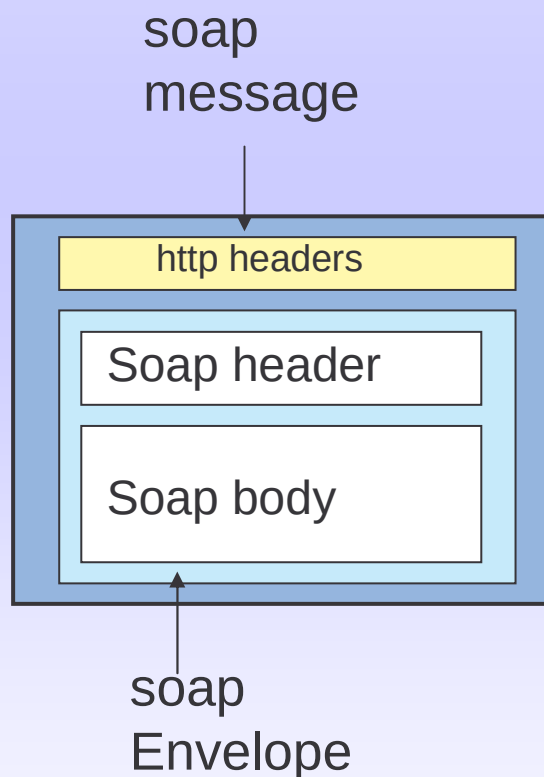
```
<methodcall>  
<methodname>weather.getWeather</methodname>  
<!-- parameter: zip code -->  
<params><param><value>10016</value></param>  
</params></methodcall>
```

Answer:

```
<methodresponse>  
<params><param><value><int>65</int></value>  
</param></params>  
</methodresponse>
```

Web Services Architecture

SOAP (Simple Object Access Protocol) (1/2)



- More complex (more expressional power) than XML-RPC (more complex tasks)
- Includes XML Namespaces and Schemas
- Structure:
 - HTTP Headers (e.g., Host: www.stock.org)
 - Soap Envelope
 - Header: for authentication, transaction, encryption...
 - Body: method name and parameters

Web Services Architecture

SOAP (2/2)

Request

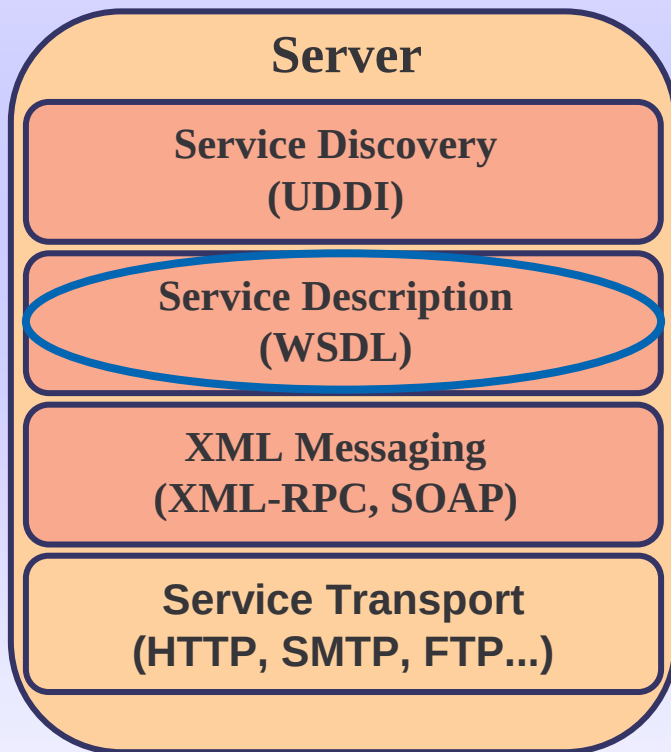
```
<soap:Envelope
xmlns:soap="..."
xmlns:xsd="... XML Schema"
xmlns:xsi="...XMLSchema-instance">
  <soap: Body>
    <ns1:getWeather xmlns:ns1="..."
      soap:encodingStyle="...">
      <zipcode xsi:type="xsd:string">
        10016</zipcode>
      </ns1:getWeather>
    </soap:Body>
  </soap:Envelope>
```

Response

```
<soap:Envelope
xmlns:soap="..."
xmlns:xsd="... XML Schema"
xmlns:xsi="...XMLSchema-instance">
  <soap: Body>
    <ns1:getWeatherResponse
      xmlns:ns1="..."
      soap:encodingStyle="...">
      <return xsi:type="xsd:int">
        65</return>
      </ns1:getWeatherResponse>
    </soap:Body>
  </soap:Envelope>
```


Web Services Architecture

WSDL (Web Service Description Language) (1/3)



- Defines the service interface for a web service (service = collection of ports)
 - What a web service can do
 - Where it resides
 - How to invoke it
- Includes:
 - Data types used by the web service (XML Schemas)
 - Publicly available function, operations used by the web service (like an API)
 - Binding information about the transport protocol used by the web service
 - Messages used by the web service

Web Services Architecture

WSDL (2/3)

```
<definitions name="WeatherService" xmlns="..." xmlns:soap="..."  
  xmlns:xsd="..." xmlns:tns=".../WeatherService.wsdl">
```

```
<!-- data elements of an operation (abstract)-->
```

```
<message name="getWeatherRequest"> <part name="zipcode"  
  type="xsd:string">
```

```
<message name="getWeatherResponse"> <part name="return"  
  type="xsd:int">
```

```
<!-- defines a web service, the operations that can perform, and the messages  
  that are involved (abstract)-->
```

```
<portType name="Weather_PortType"> <operation name="getWeather">
```

```
<input message="tns:getWeatherRequest"/>
```

```
<output message="tns:getWeatherResponse"/>
```

```
</operation></portType>
```

Web Services Architecture

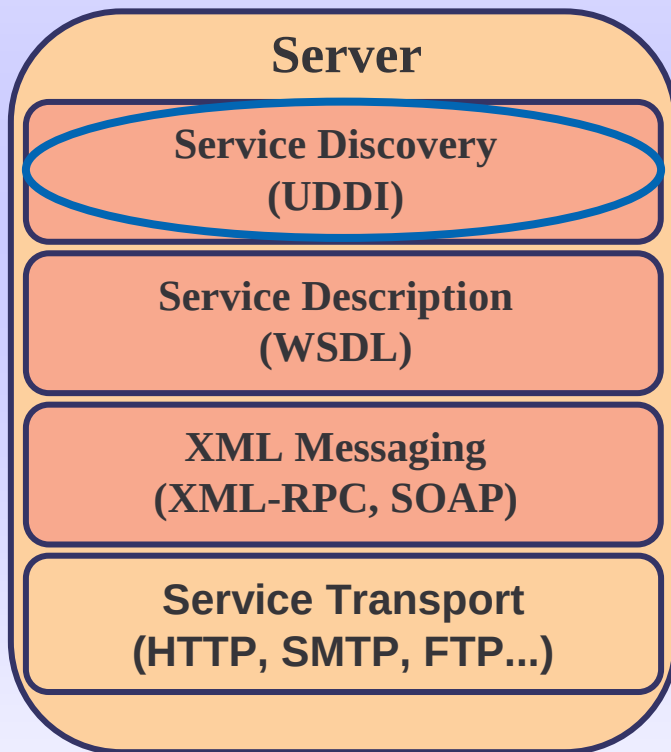
WSDL (3/3)

```
<!-- message format and protocol details (concrete) for each port type -->
<binding name="Weather_Binding" type="tns:Weather_PortType">
  <soap:binding style="rpc" transport=".../soap/http"/>
  <operation name="getWeather">
    <soap:operation soapAction=""/>
    <input> <soap:body encodingStyle="..."></input>
    <output><soap:body encodingStyle="..."></output>
  </operation>
</binding>

<!-- collection of related endpoints-->
<service name="Weather_Service"> <documentation>...</documentation>
  <!-- single endpoint -->
  <port binding="tns:Weather_Binding" name="Weather_Port">
    <soap:address location="...">
  </service>
</definitions>
```

Web Service Architecture

UDDI (Universal Description, Discovery, and Integration)



- Represents the discovery/publishing layer for the web services
- Provides a registry mechanism for clients and servers to find each other
- Three categories: white, yellow, and green pages

White

General information about a specific company (e.g., business name)

Yellow

General classification data for either a company or the offered services

Green

Technical info about service (how to invoke their services) – a point to WSDL
Specify how to bind to a service provider.

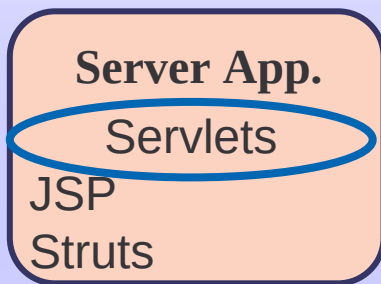
Server Applications

Server App.

Servlets
JSP
Struts

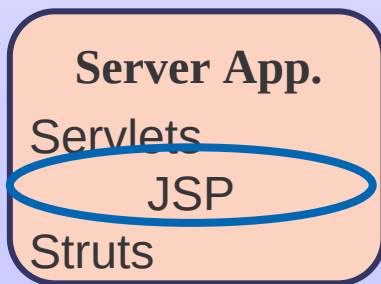
- Goal: Development of interactive server side applications
- Two major competitors:
 - Microsoft: .NET
 - Sun: Java 2 Enterprise Edition (J2EE)
- Java Benefits:
 - Free (open standard), no tied to any vendors
 - Security
 - Sandbox security
 - Secure Runtime model (bytecode verification...)
 - Applets (on client-side) are also Java
- J2EE components (functional software unit)
 - Application clients and applets (on the client)
 - Servlet and JavaServer Pages (JSP) (on the server)
 - Enterprise Java Beans (EJB)=business components

Server Applications Servlets



- Java class that dynamically process request and produce response following HTTP pattern
- HTTP:
 - Stateless: client request, server responds
 - Request:
 - Get: simple request (just getting data)
 - Post: bigger request (anything: storing data, ordering a product)
 - ...
 - Response: a code (e.g., 200 OK)
- HttpServlet methods: init, doGet, doPost, getServletName, destroy
- Sessions (maintained by HttpSession object) using Cookies

Server Applications JSP



- Text based documents (HTML or XML templates)
- Java code is embedded
 - Expressions: `<%= expression %>`
 - `<%= new java.util.Date(); %>`
 - Declarations: `<%! declaration %>`
 - `<%! private int number = 0; %>`
 - Statements: `<% statement %>`
 - `<% out.println(++number); %>`
 - JSP directives: `<%@ directive %>`
 - information for translator
 - `<%@ page import="java.util.*" %>`
 - `<%@ include file="hello.jsp" %>`
- Execute as servlets
 - JSP are translated into Servlets
 - All Servlets features are directly available
- More natural approach to create static content ?

Server Applications (Example Servlet & JSP)

Servlet handles contents and logic

```
public class MyServlet extends
    HttpServlet{

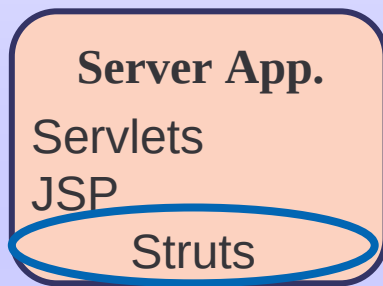
    public void doGet(...) throws ...{

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><head><title> JSP
            </title>");
        out.println(" </head>");
        out.println(" <body>
            <h1>HELLO!</h1>");
        out.println("Last updated" + new
            java.util.Date());
        out.println(" </body></html>");
    }
}
```

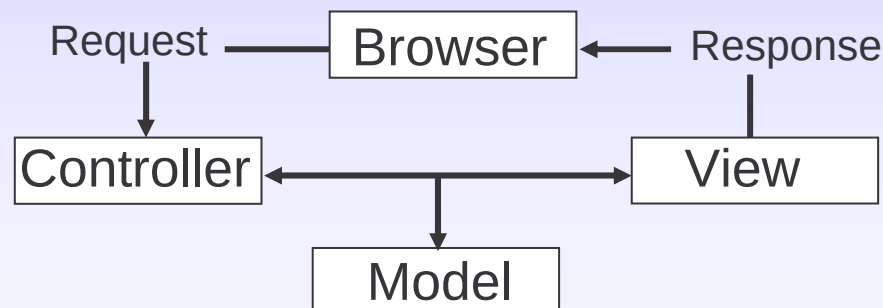
JSP for presentation

```
<html><head><title> JSP </title>
</head>
<body> <h1>HELLO!</h1>
Last updated: <%= new
    java.util.Date() %>
</body></html>
```

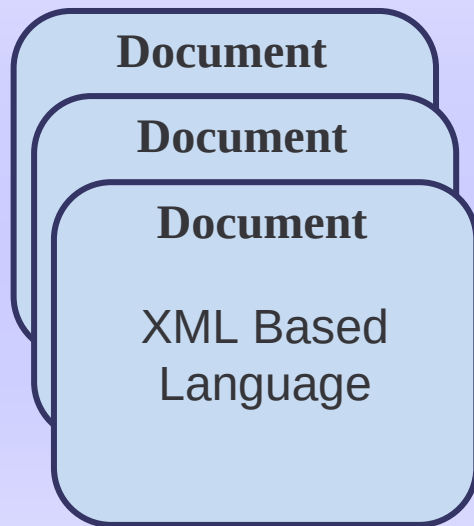

Server Applications Struts



- Open Source framework for building web applications
- Following the Model-View-Controller (MVC) architecture
 - Model: data and operations
 - Implemented as Java classes
 - View: presentation
 - Implemented as JSP
 - Controller: logic, receives request, update the model, and delegates to views
 - Implemented as a single servlet



XML Based Documents



- HTML & XHTML
- Multimedia
 - SMIL, Timesheets
- User Interface
 - XForms, XIML
- Vector Graphics
 - SVG
- Voice
 - VoiceXML

XML Based Languages

HTML & XHTML

HTML

- HTML 4.01: (24 Dec. 1999) W3C Recommendation
- Lingua franca for publishing hypertext on the WWW.
- Non-proprietary
- Can be created by a wide range of tools:
 - Text editors
 - Authoring tools
- All kind of features (mixed together):
 - UI components
 - Fonts
 - Lists

XHTML

- XHTML 1.0 (26 Jan. 2000, revised 1 Aug. 2002) W3C Recommendation
- XHTML 2.0: (22 July 2004) W3C Working Draft
- Reformulation of HTML 4 in XML
- Intention
 - To only describe the structure of the document (CSS formatting)
- XHTML 1.0
 - Well formed documents
 - Proper nesting
 - ...
- XHTML 2.0
 - No backwards compatible
 - Reduces scripting
 - Includes XForms and XML Events

XML Based Languages Multimedia

SMIL

- SMIL 2.0 (07 Aug. 2001) W3C Recommendation
- Easy to write, like HTML
- Doesn't define media formats, only integrates them
 - `<img>`, `<video>`, `<audio>`
- Defines the spatial and temporal dimensions of the document
- Limited Interaction
 - `<a>`, `<area>`: for links
- Absolute Layout

Timesheets

- Similar to CSS, but for temporal dimension
- Document composed of:
 - Content: XHTML
 - Formatting: CSS
 - Timing: Timesheets
- Similar syntax than SMIL

`<time><par>`

`<item select="#isect1"
dur=10s>`

`<item select="#isect2"
dur=10s>`

`</par></time>`

XML Based Languages User Interface

XForms

- XForms 1.0 (14 Oct. 2003) W3C Recommendation
- Next generation of web forms
- Not intended as a self-standing document type
- Uses host language for the document layout (e.g., XHTML, SMIL)
- Advances user interface features:
 - text input, select one, select many, submit
- User input can be validated in the client-side
- Calculations are done, as well, in the client side

XUL

- XML User Interface Language
- Only supported in Mozilla and Netscape 6 (or later) browsers
- Only for window-based graphical UI (mobile phones?)
- Abstraction only at the platform level (not at the UI level, voice?)
- It separates:
 - Client application definition and programmatic logic
 - Presentation (using CSS)
 - Language-specific text labels
- Look & feel changed as wished
- Interaction achieved by scripting
- Interface elements: windows, menubar, scrollbar

XML Based Languages

Vector Graphics, Voice

SVG

- SVG 1.0 (04 Sept. 2001) W3C Recommendation
- SVG 1.1 (14 Jan. 2003) W3C Recommendation
- Describes vector-based graphics for the Web (no pixel based)
 - Shapes (e.g., lines & curves)
 - Images
 - Text
- Drawings can be
 - Interactive (e.g., Mouse clicked)
 - Animated (e.g., Change location)

VoiceXML

- VoiceXML 2.0 (16 Mar. 2004) W3C Recommendation
- Creation of audio dialogs (user interfaces)
- Input
 - Speech Recognition and/or touch tone (keypad)
- Output
 - Pre-recorded audio and Text-to-Speech Synthesis (TTS)
- Describes:
 - Spoken prompts: synthetic speech
 - Recognition of spoken words and touch tone key presses (fields)
 - Control of dialog flow (menu, form that can be submitted to server)
 - Telephony control (call transfer)

Terminals & Browsers

Desktop

**Desktop
Browser**

**Embedded
Browser**

**Voice
Browser**



<http://www.xsmiles.org>



<http://www.mozilla.org>



<http://www.microsoft.com/windows/ie/>



<http://www.opera.com/>



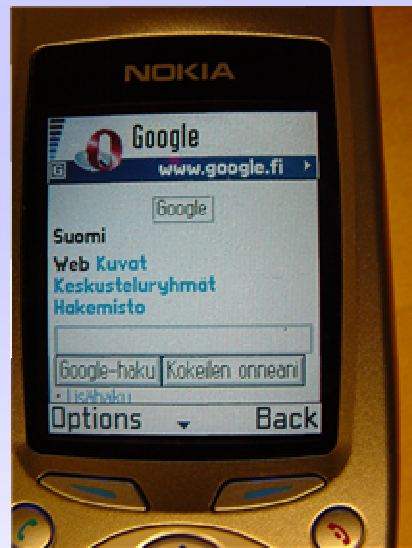
<http://home.netscape.com/>

Terminals & Browsers Embedded

Desktop
Browser

Embedded
Browser

Voice
Browser



Mobile Phone



Espial
Browser



Web TV

Terminals & Browsers Multimodal

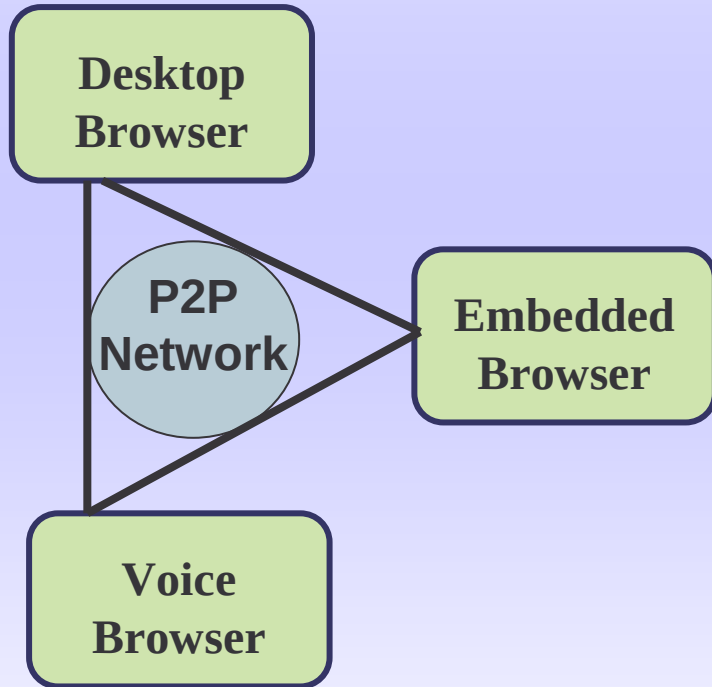
**Desktop
Browser**

**Embedded
Browser**

**Voice
Browser**

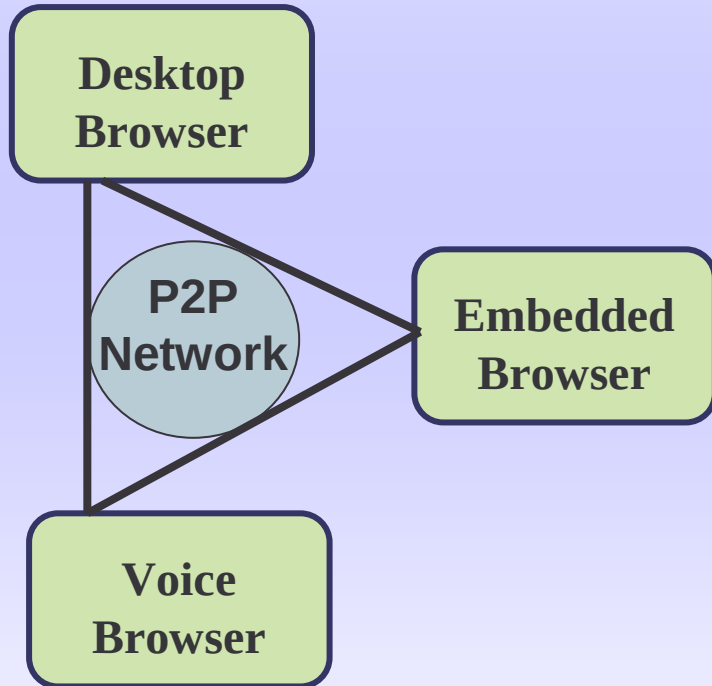
- Information taken from:
<http://www.w3.org/WAI/References/Browsing>
- Reasons:
 - People with disabilities (e.g., hearing or visual)
 - Context of use (e.g., car)
 - ...
- Solutions:
 - Speech input/output
 - Braille displays
 - Screen magnification
 - Use of Keyboard instead of mouse
- Examples:
 - BrailleSurf: internet browser that shows information in textual form (speech output, displayed in a Braille bar)
 - ConversaWeb: web surfer program that you can control using voice
- Screen-Readers
 - Programs that translate the screen presented by the OS in speech output or Braille (can be used for any application)

P2P Architecture



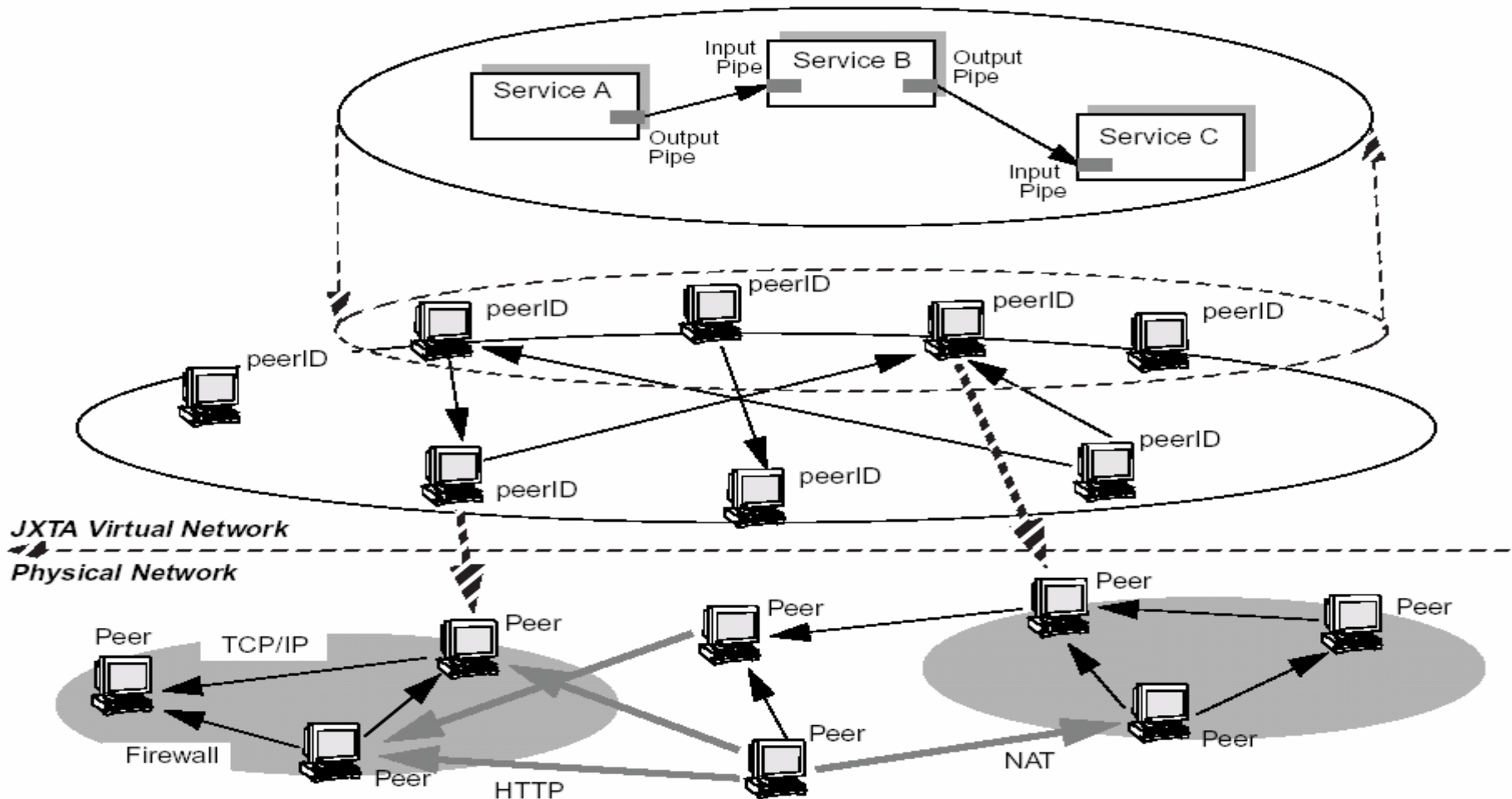
- Client/Server architecture does not really work for collaboration (e.g., file sharing) or direct communication
- Relies in a centralised process to maintain:
 - Information of the session
 - Information about members
- Basic problems of Client/Server:
 - Processes could be bottleneck
 - Too many demands -> denial of service
- Why to use P2P?
 - End devices capabilities are not fully used (nowadays quite powerful)
 - Fixed price connections do not take into account the usage of the net

P2P Architecture



- Systems are decentralised and networked
- There is not server, all machines are equal (peers):
 - Direct connection between peers
 - Nodes join and leave as wished
 - Some implementations use stronger peers (super-modes)
- Popular as desktop applications:
 - ICQ, AIM, Gnutella, SETI@home, eDonkey, Kazaa
- Applications include:
 - chats, instant messaging, file sharing (music, movies, images) and CPU sharing (remote processing)

P2P Network (e.g., JXTA)

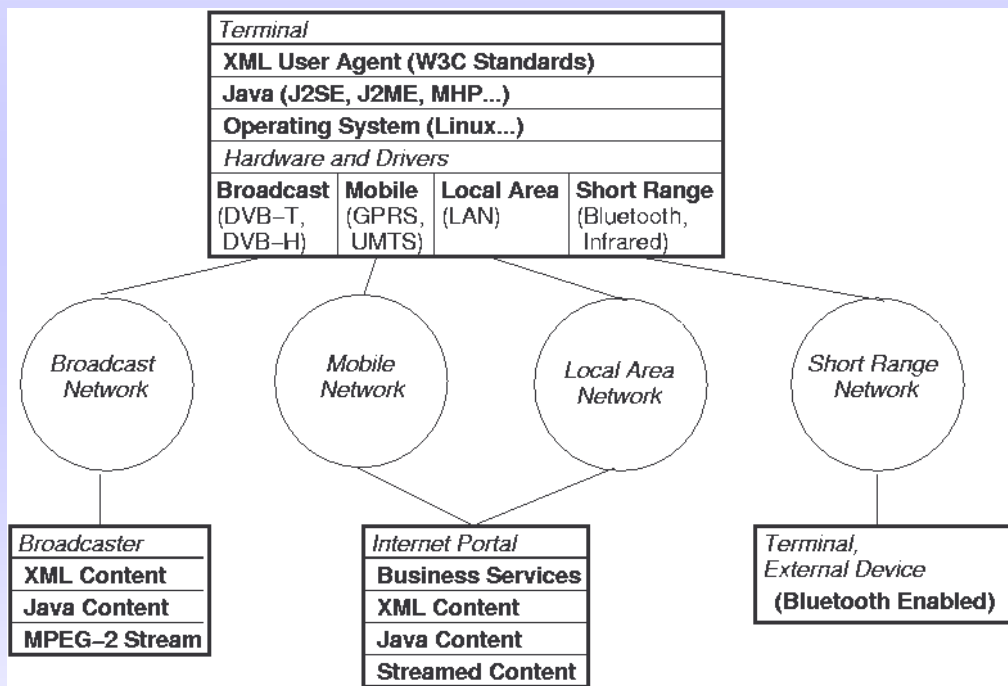


Outline - Part II

Graphics Systems in Embedded Devices

- Introduction
 - Multimedia Terminals Chaos
 - Digital Television
- Graphics Architecture
 - From Windows to Scene Based Graphics Architectures
- Reference Implementation
 - Ubik Overview
 - Ubik Architecture
 - Ubik Screenshots
 - High Level Languages Integration
 - High Level Languages Profiles
 - High Level Languages Screenshots

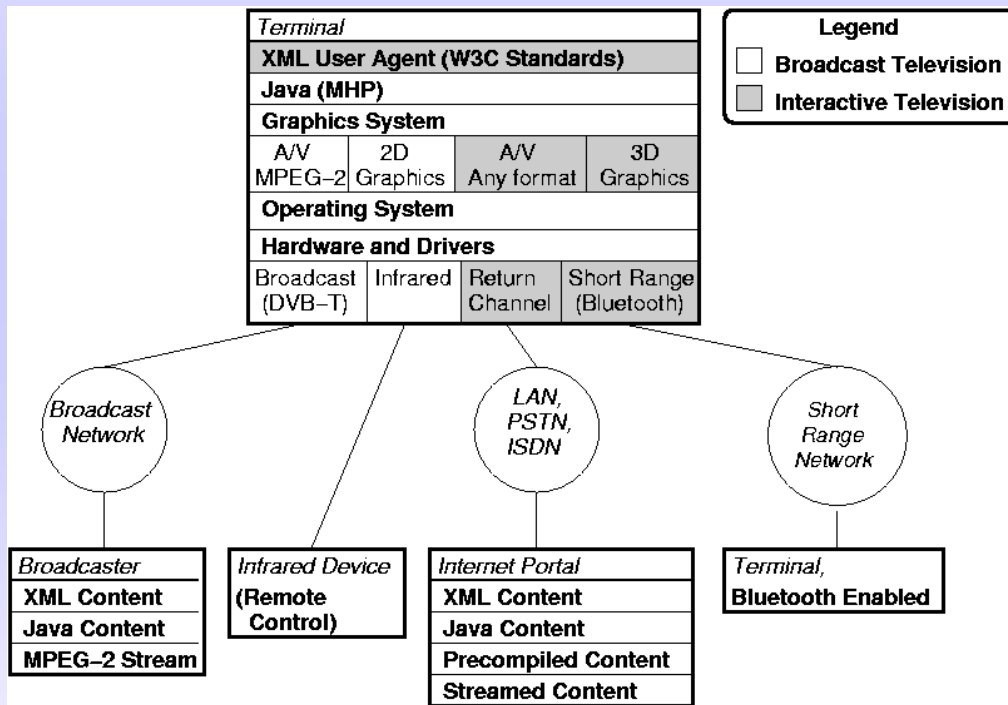
Introduction



- Variety of multimedia devices
 - Desktop (PCs)
 - Non-desktop (STB)
- Number of networks
 - Broadcast
 - Mobile
- Diversity of Content
 - MPEG-2
 - Java
 - XML based

Introduction

Digital Television (1/2)



- Digital TV receivers today
 - Starting to show technological maturity
- Digital TV receivers tomorrow
 - Evolution, so different configurations depending on the targeted group
 - Broadcast
 - Only uses broadcast network
 - Interactive
 - Uses, as well, interaction channel

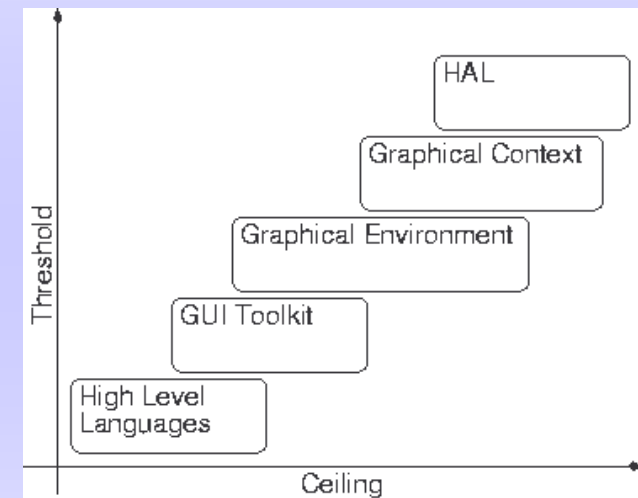
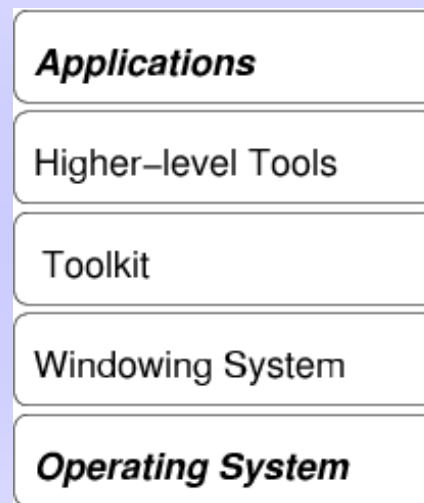
Introduction

Digital Television (2/2)

- Standards = Horizontal Market
 - Applications and decoders can be developed by different developers and manufacturers.
- MHP (Multimedia Home Platform)
 - Platform independent middleware between applications and receivers
 - Defines two languages
 - DVB-J: Java Based Language
 - DVB-HTML: Complex XML Based Language

Graphics Architecture

- Lack of pointing device
- Seameless integration of video, 2D and 3D objects
- Screen composed of multimedia objects
- A layered architetcure, so developers can implement at any level
- Solutions such as X-Windows are too big

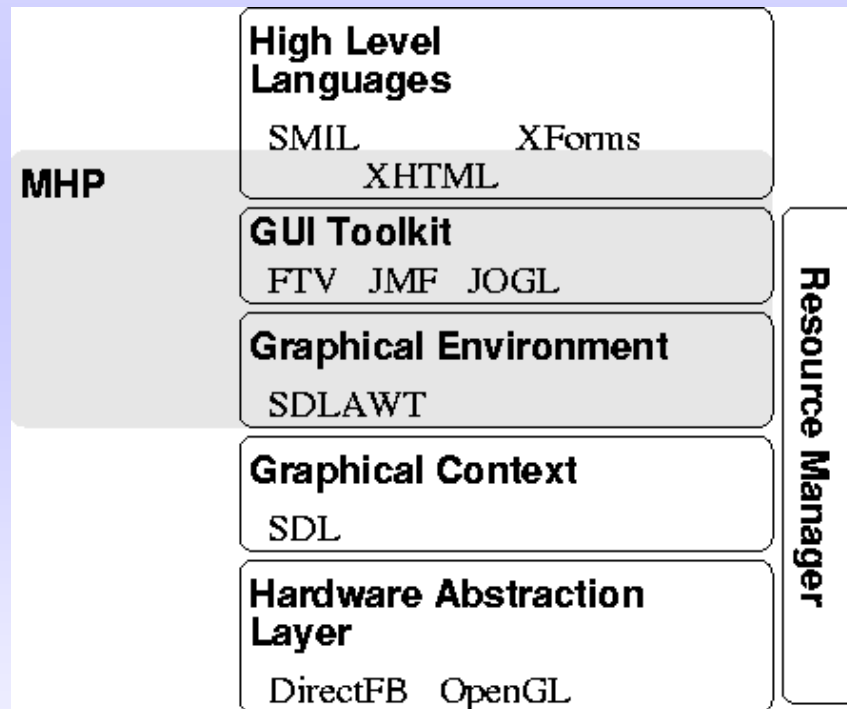


- HAL: an unified way to access hardware (hardware acceleration)
- Graphical Context: cross-platoform abstraction of the rendering region
- Graphical Environment: means to control different contexts
- GUI Toolkit: "ready-made" user interface widgets
- HLL: to develop simple services

Reference Implementation Ubik Overview

- Configurable, open, and extensible digital television receiver prototype
- Platform that allows the study of possible extensions to MHP
- Topics
 - DVB-T: basic A/V support (MPEG-2) and video player (avi, mov...)
 - 3D graphics support: based on OpenGL
 - Java support: applications such as Teletext or Navigator
 - XML support: advanced functionality (X-smiles)
- Output (2004)
 - Two journal papers
 - One conference paper
 - One doctoral dissertation
 - Collaboration with OpenMHP open source project

Reference Implementation Ubik Architecture



- DVB-T reception and visualisation of the A/V stream
- HAL: DirectFB and OpenGL
- Resource Manager: to control the different processes
- Graphical Context: SDL
- Graphical Environment: SDLAWT (java.awt)
- GUI Toolkit
 - FTV: 2D widgets
 - JMF: other video than A/V
 - Java OpenGL: 3D Graphics
- HLL: X-smiles (XHTML, SMIL, XForms)

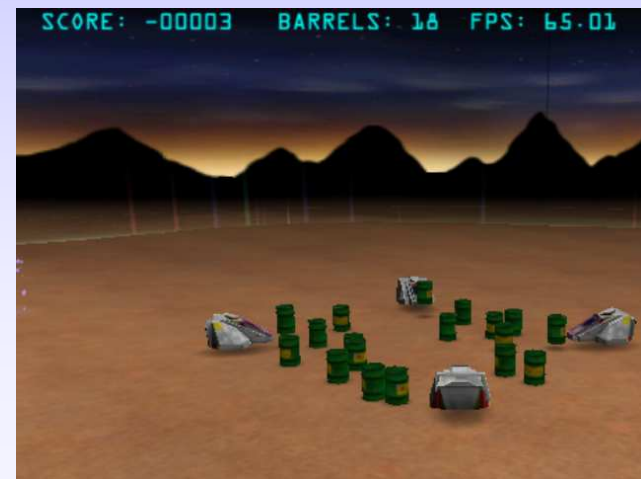
Reference Implementation Ubik Screenshots (1/2)

- Example composition of scenes:
 - 3D graphics object
 - A/V Stream
- Performance
 - Around 60 FPS



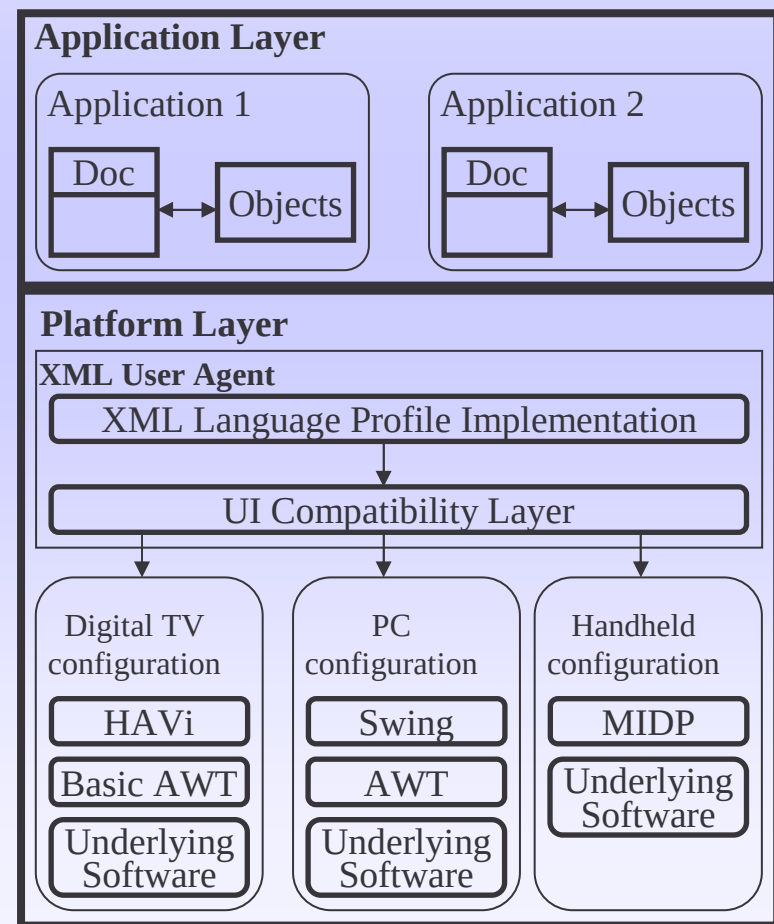
Reference Implementation Ubik Screenshots (2/2)

- Example native 3D Graphics
 - Some games downloaded from a portal



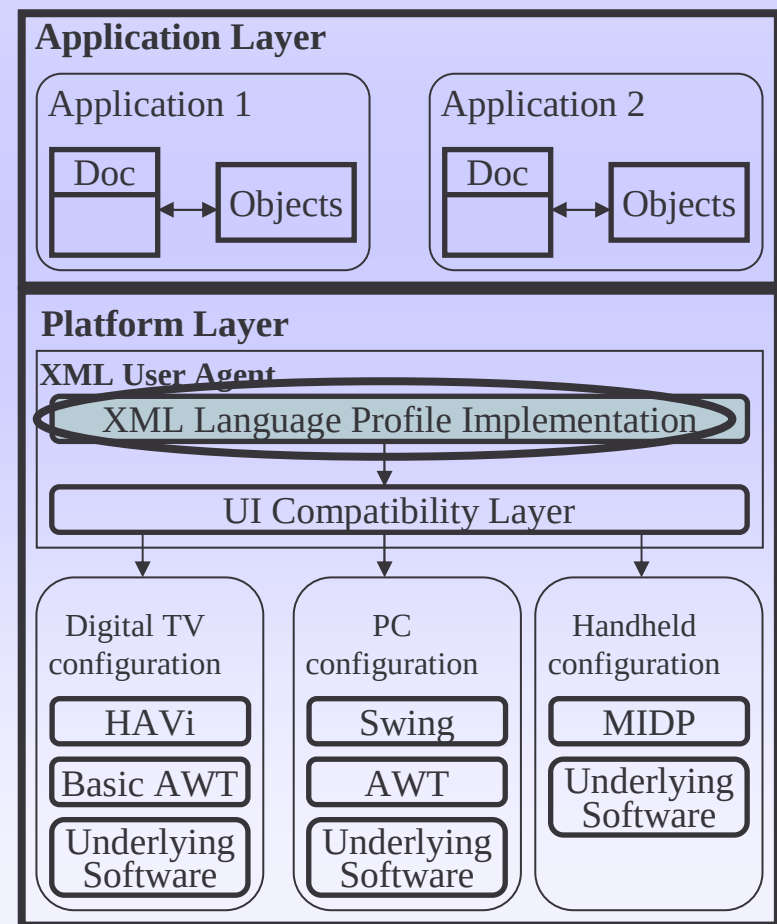
Reference Implementation High Level Languages Integration (1/6)

- Application Layer:
 - XML languages used for implementing interactive multimedia applications
- Platform Layer:
 - User Agent supports the language profile
 - UI Compatibility Layer (Component Factory)
 - Specific Device configuration



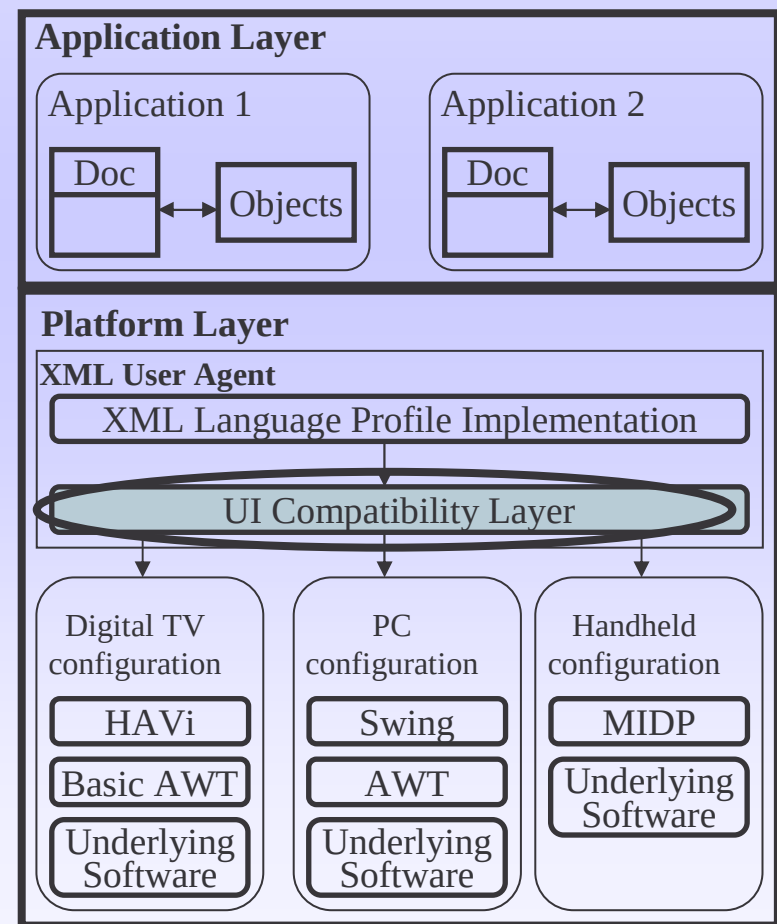
Reference Implementation High Level Languages Integration (2/6)

- Temporal Dimension
- Spatial Layout / Flow Layout
- Multimedia Objects Support
 - Continuous
 - Discrete
- User Interaction
 - Links
 - Validated Entry
 - Submission (server-side)

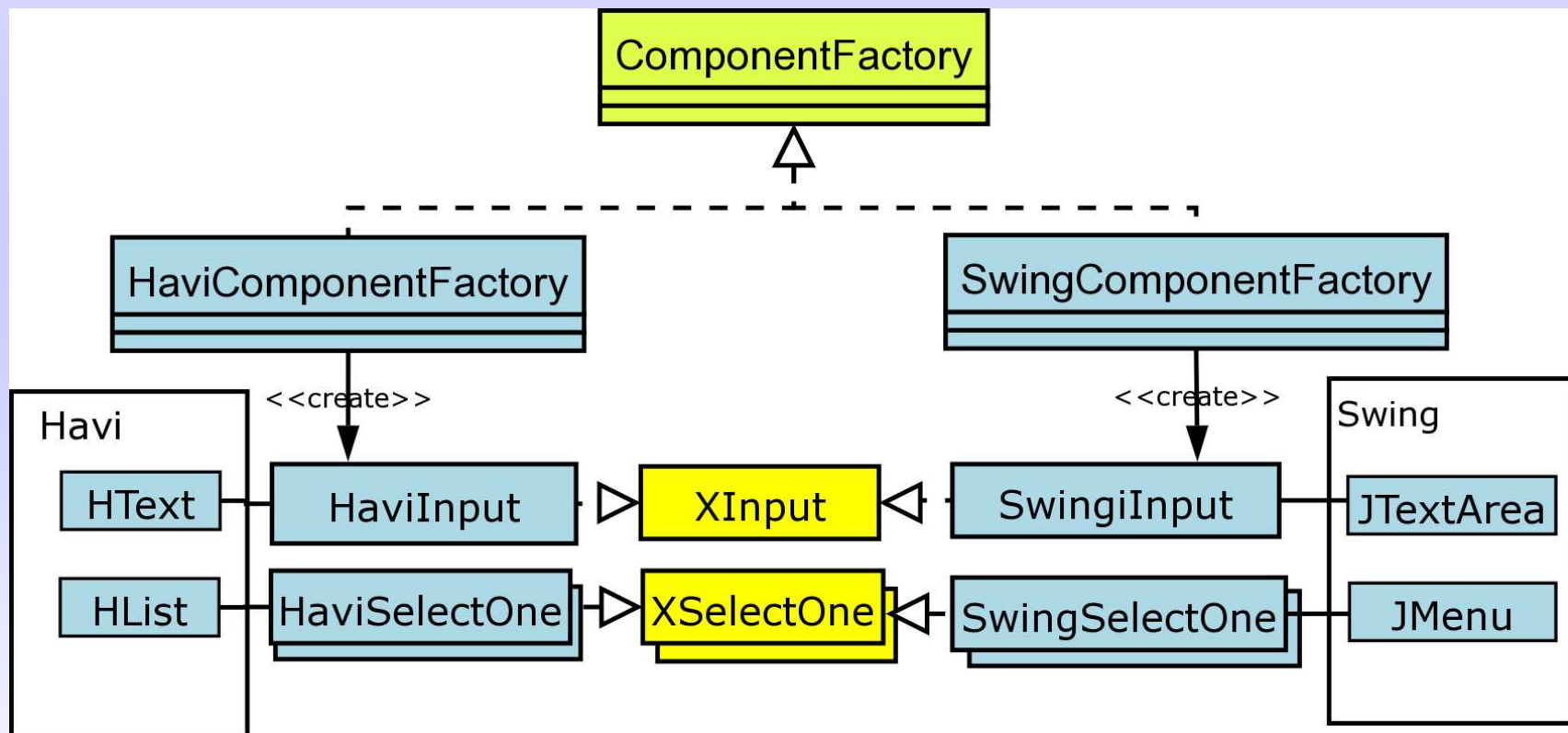


Reference Implementation High Level Languages Integration (3/6)

- Platform uses Java
- Most platform independent option
- The actual APIs for user interface development differ between multimedia terminals:
 - Digital TV: HAVi
 - PC: Swing
 - Handheld: MIDP



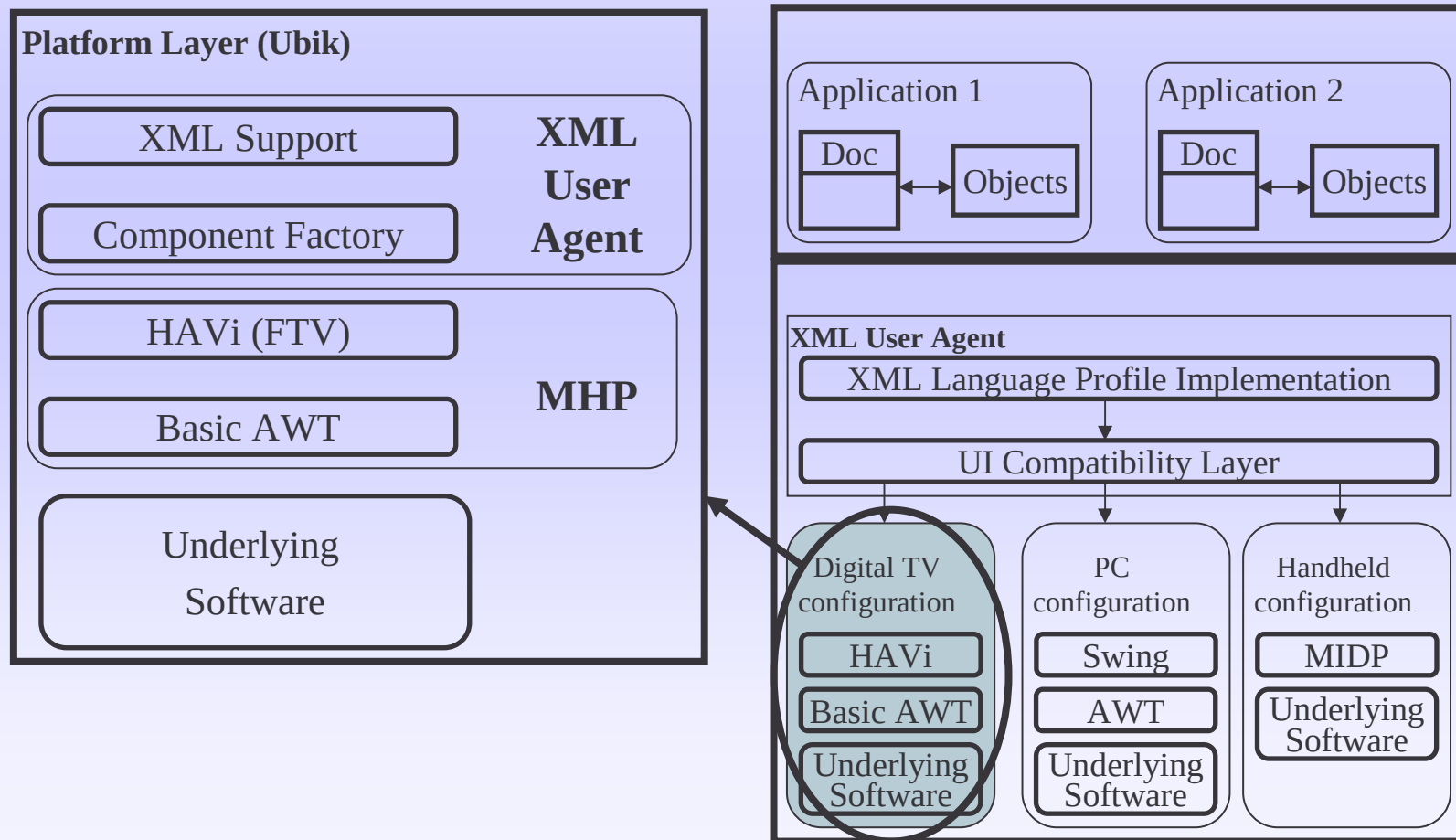
Reference Implementation High Level Languages Integration (4/6)



Reference Implementation High Level Languages Integration (5/6)

XForms Control	AWT Widget	Swing Widget	HAVi Widget
select1 & select	List	JList	HListGroup
trigger	Button	JButton	HTextButton
submit	Button	JButton	HTextButton
label	TextField	JTextField	HStaticText
textarea	TextArea	JTextArea	HText
input + xsd:string	TextField	JTextField	HSingleLineEntry
input + xsd:date	TextField	JCalendar	HSingleLineEntry
input + xsd:boolean	CheckBox	JCheckBox	HToggleButton

Reference Implementation High Level Languages Integration (6/6)



Reference Implementation High Level Languages Profiles

Internet Access

- Requirements:
 - Access to Web pages
 - Limited navigation
 - Images, text
 - Small size
- Proposed profile:
 - XHTML 1.0
 - CSS

High End

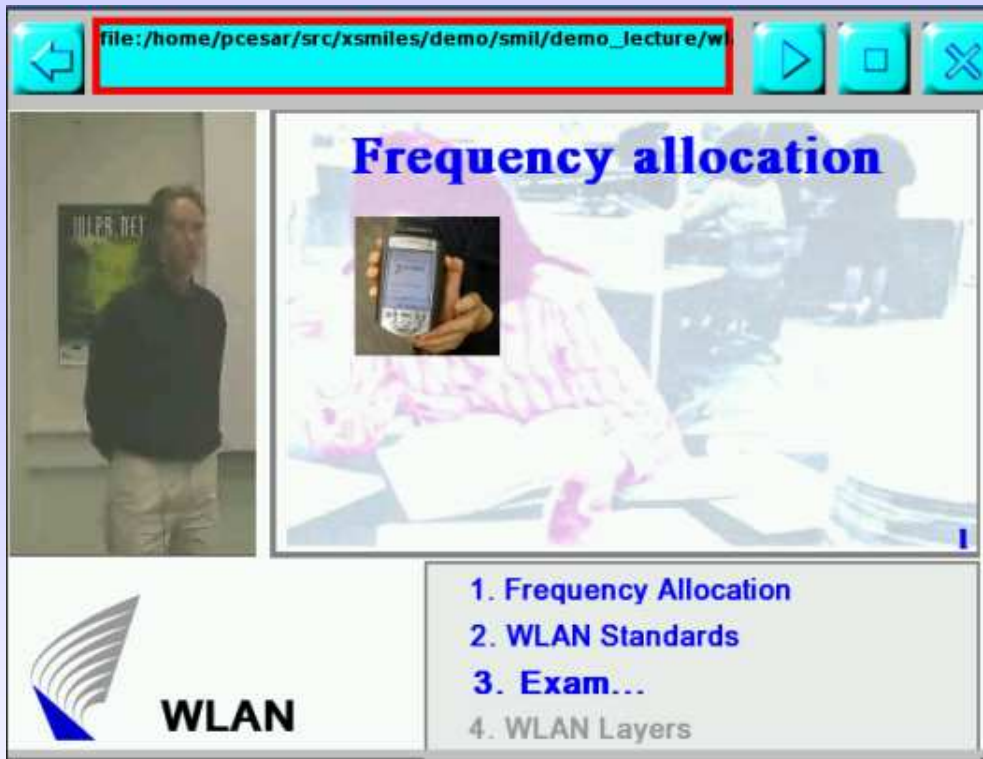
- Requirements
 - Access to complex Web applications
 - Interactive
 - Temporal Synchronisation
 - Video and audio
 - Bigger size
- Proposed profile:
 - XHTML 2.0 + Timesheet
 - SMIL + XForms

Reference Implementation High Level Languages Screenshots (1/2)



- XML Based Document
- Profile:
 - Interactive Access
- Target:
 - Information Services (e.g., Teletext)
- Languages:
 - XHTML 1.1
- Interaction:
 - Navigation
- Multimedia Objects:
 - Images
 - Text

Reference Implementation High Level Languages Screenshots (2/2)



- XML Based Document
- Profile:
 - High End
- Target:
 - Complex Applications (e.g., E-learning Portal)
- Languages:
 - SMIL (or XHTML+Timesheets)
 - XForms
- Interaction:
 - Buttons/Selections...
 - Navigation
- Multimedia Objects:
 - Images/Text
 - Video/Audio

Outline - Part III

Experiences from a PhD Student (myself)

- Little Biography
- Before PhD (2001)
- Starting (2002)
- Mid Way (2003)
- Ending (2004)
- Milestones
- Conclusions

Little Biography

- Ms. Sc. from Universidad Politecnica de Madrid (February 2002)
- Master's Thesis: HAVi Components in Digital Television under supervision of Petri Vuorimaa
- May 2000 – Dec 2001: Assistant Researcher (Future TV Project)
- Jan 2002 – Dec 2002: Researcher (Brocom Project)
- Jan 2003 – date: Project Manager and Researcher (Brocom Project)

Before PhD [Master's Thesis]

- Responsibilities
 - Port a number of Digital Television services to Kaffe
 - Develop HAVi widgets
 - Write Master's Thesis
- Publications:
 - C. Peng, P. Cesar, and P. Vuorimaa, "Integration of applications into digital television environment," *Proceedings of the International Workshop on Multimedia Technology, Architecture and Application*, 2001
 - G. Sivaraman, P. Cesar, and P. Vuorimaa, "System software for digital television applications," *Proceedings of the IEEE International Conference on Multimedia and Expo, ICME2001*, 2001

Starting (2002) [Looking for a Problem]

- Responsibilities
 - Researcher in Brocom project
 - First Ideas: UI Frameworks / Alpha channel / Video Provider
- Publications
 - P. Cesar and P. Vuorimaa, "A graphical user interface framework for digital television," *Proceedings of the 10th International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision, WSCG'2002*, 2002
 - K. Pihkala, P. Cesar, and P. Vuorimaa, "Cross-platform SMIL player," *Proceedings of the 1st IASTED International Conference on Communications, Internet and Information Systems, CIIT2002*, 2002
- Failures
 - ICME Paper

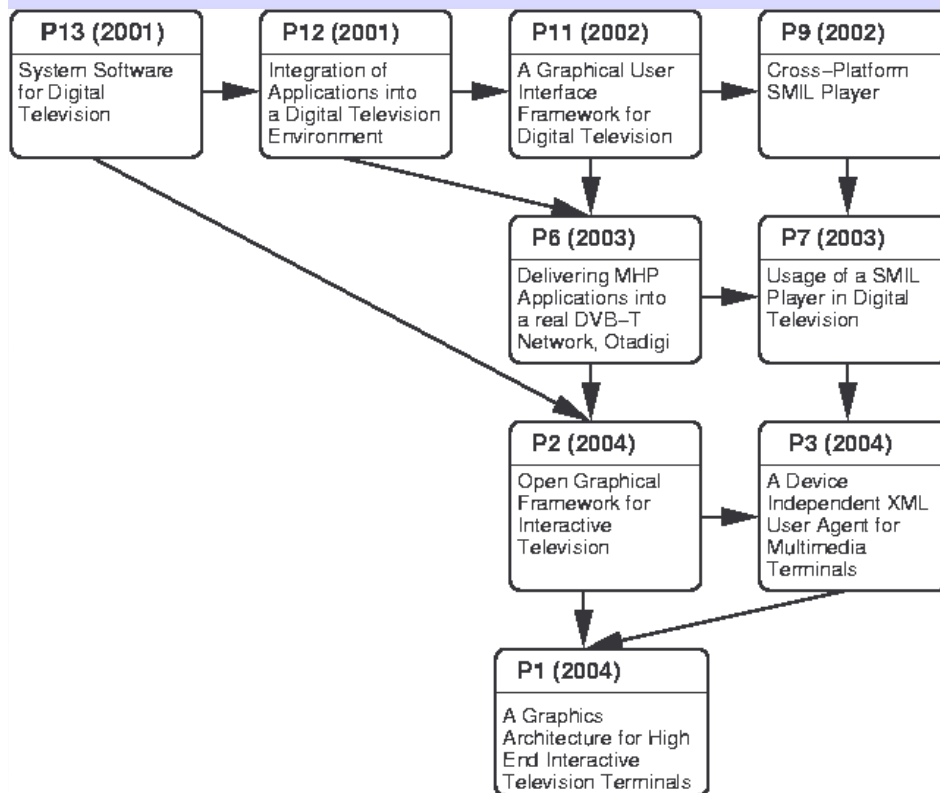
Mid Way (2003) [Researcher]

- Responsibilities:
 - Otadigi (Digital TV big picture)
 - Project Manager (Research Group big picture)
 - Development of Ubik
- Publications:
 - C. Herrero, P. Cesar, and P. Vuorimaa, "Delivering MHP applications into a real DVB-T network, Otadigi," Proceedings of the *6th International Conference on Telecommunications in Modern Satellite, Cable and Broadcasting Services, TELSIKS2003*, 2003
 - P. Cesar, J. Vierinen, and P. Vuorimaa, "Open graphical framework for interactive TV," Proceedings of the *IEEE 5th International Symposium on Multimedia Software Engineering, MSE2003*, 2003
 - J. L. Lamdon, P. Cesar, C. Herrero, and P. Vuorimaa, "Usages of a SMIL player in digital television," Proceedings of the *7th IASTED International Conference on Internet and Multimedia Systems and Applications , IMSA 2003*, 2003
 - ...

Finish (2004) [Maturity (Really?)]

- Responsibilities
 - Project Manager
 - Write Thesis
 - Development of Ubik
- Publications
 - P. Cesar, J. Vierinen, and P. Vuorimaa, "Open graphical framework for interactive TV," Accepted to the *International Journal on Multimedia Tools and Applications*. Kluwer Academic
 - M. Honkala, P. Cesar, and P. Vuorimaa, "A device independent XML user agent for multimedia terminals," Accepted to the *IEEE 6th International Symposium on Multimedia Software Engineering, MSE2004*, 2004
 - P. Cesar, J. Vierinen, and P. Vuorimaa, "A Graphics Architecture for High End Digital Television Terminals," Submitted to ACM Transactions on Computing Multimedia, Communications, and Applications
- Failures
 - WWW paper

Milestones



- P13: general study on the system software for DTV
- P12: description of a system model to run different DTV applications
- P11: Graphical User Interface Toolkit for DTV
- P9: portable SMIL player running on different devices
- P7: possible use case scenarios for SMIL in DTV
- P6: description of two services transmitted over a DTV network
- P3: definition of an XML based profile for developing interactive multimedia services (SMIL+XForms)
- P2: layered graphics architecture for non-desktop devices
- P1: extension to P8 with detailed study on graphics architectures alternatives. Complete reference implementation

Conclusion

- Stair paradigm: you will go through a researcher process (Starting-Researcher-Maturity) and shouldn't worry about it
 - PhD = Mature Researcher
 - Don't try to force the speed of the process
- Research Question, Assumptions, and Justification of the Study
 - I am still writing them today
- What worked
 - Reading a lot (and i mean really a lot) [Not only technical papers]
 - Applying for grants (force to rethink the research plan)
 - Taking responsibilities
 - Talking a lot with colleagues, professors...
 - Taking at least 8 credits per term
 - Failures are actually wins (you learn so much and think deeper)
 - Write a lot
 - Keep diary and notebook (organise your time)